容量制約付き施設配置問題 (CFLP)

AMPL / amplpy による厳密解法と Timefold 解の比較・検証

数理最適化演習

October 27, 2026

lecture2026

アジェンダ

問題の概要と設定

記号と数理定式化

AMPL コードの解説

Python (amplpy) 実装の解説

最適化結果と Timefold 解の比較

まとめ

問題の概要と設定

施設配置問題（Timefold quickstarts）の概要

- 問題背景:
  - 複数の施設候補地から最適な場所を選定（開設）し、顧客の需要を充足させる代表的組合せ最適化問題（Capacitated Facility Location Problem: CFLP）。
- データセット（ロンドン: 施設 30・顧客 60）:
  - 施設候補数: 30 箇所（各施設に固有の開設費用 f_j、容量 $C_j = 150$）
  - 顧客数: 60 箇所（各顧客の需要 $d_i = 15$、合計需要 = 900）
- 最適化のトレードオフ:
  - 開設費用 (Setup Cost): 開設する施設数を絞るほど安くなる。
  - 輸送・移動距離 (Distance): 施設を分散配置するほど顧客への距離が短くなる。
- 距離計算式 (Timefold 準拠):

$$dist(i,j) = \left\lceil \sqrt{(\mathrm{lat}_i - \mathrm{lat}_j)^2 + (\mathrm{lon}_i - \mathrm{lon}_j)^2} \times 111{,}000 \right\rceil \quad [\mathrm{m}]$$

記号と数理定式化

記号の定義（Notation）

集合 (Sets)

- $I = \{1, \ldots, |I|\}$: **顧客集合** ($|I| = 60$)
- $J = \{1, \ldots, |J|\}$: **施設候補集合** ($|J| = 30$)

重み係数 (Weights)

- $w_{\text{setup}} = 2$: **施設開設費用の重み**
- $w_{\text{dist}} = 6$: **移動距離の重み**

パラメータ (Parameters)

- d_i: **顧客** i **の需要量** ($= 15$)
- C_j: **施設** j **の受入容量** ($= 150$)
- f_j: **施設** j **の固定開設費用**
- $dist_{ij}$: **顧客** i **と施設** j **間の距離** (m)

決定変数 (Decision Variables)

- $y_j \in \{0, 1\}$: **施設** j **を開設するとき** 1, **他は** 0
- $x_{ij} \in \{0, 1\}$: **顧客** i **を施設** j **に割当てる時** 1, **他は** 0

数理定式化（0-1 整数線形計画問題）

目的関数：加重総ペナルティの最小化

$$\min \quad \sum_{j \in J} w_{\text{setup}} f_j y_j + \sum_{i \in I} \sum_{j \in J} w_{\text{dist}} dist_{ij} x_{ij}$$

制約条件

$$\sum_{j \in J} x_{ij} = 1 \qquad \forall i \in I \quad \text{（単一割当制約）} \tag{1}$$

$$\sum_{i \in I} d_i x_{ij} \leq C_j y_j \qquad \forall j \in J \quad \text{（施設容量制約）} \tag{2}$$

$$x_{ij} \leq y_j \qquad \forall i \in I, \forall j \in J \quad \text{（開設連動制約 / 強定式化）} \tag{3}$$

$$y_j \in \{0,1\}, \quad x_{ij} \in \{0,1\} \qquad \forall i \in I, \forall j \in J \quad \text{（0-1 整数変数）}$$

各制約の数理的・実務的意味

- **単一割当制約** (1):
  - 各顧客 i の需要はちょうど 1 つの施設から供給される（分割供給を禁止）。
- **施設容量制約** (2):
  - 開設された施設 ($y_j = 1$) の需要合計は最大容量 C_j 以下。
  - 未開設の施設 ($y_j = 0$) には需要を一切割り当てられない。
- **開設連動制約 (Linking Constraint)** (3):
  - 理論的には制約 (2) から導出可能だが、加えることで LP 緩和（線形緩和）の実行可能領域を大幅に引き締める（Strong Formulation）。
  - 分枝限定法（Branch and Bound）における下界値が格段に良くなり、探索ノード数を大幅に削減。

AMPL コードの解説

AMPL モデル定義（Model Formulation）

```
set CUSTOMERS;   # 顧客集合 I
set FACILITIES;  # 施設候補集合 J

param demand{CUSTOMERS};          # 需要量 d_i
param capacity{FACILITIES};       # 容量 C_j
param setup_cost{FACILITIES};     # 開設費用 f_j
param dist{CUSTOMERS, FACILITIES};# 距離 dist_ij
param w_setup;                    # 開設重み (2)
param w_dist;                     # 距離重み (6)

var y{FACILITIES} binary;             # 施設開設決定変数 y_j
var x{CUSTOMERS, FACILITIES} binary; # 顧客割当決定変数 x_ij

# 目的関数：加重総コスト最小化
minimize TotalCost:
    sum{j in FACILITIES} w_setup * setup_cost[j] * y[j]
    + sum{i in CUSTOMERS, j in FACILITIES} w_dist * dist[i, j] * x[i, j];
```

AMPL 制約条件定義（Constraints）

```
# 1. 単一割当制約: 各顧客は必ず1つの施設に割り当て
s.t. AssignOnce{i in CUSTOMERS}:
    sum{j in FACILITIES} x[i, j] = 1;

# 2. 容量制約: 各施設に割り当てられた需要合計 <= 容量 * y[j]
s.t. Capacity{j in FACILITIES}:
    sum{i in CUSTOMERS} demand[i] * x[i, j] <= capacity[j] * y[j];

# 3. 強定式化連動制約: y[j] が 0 ならば x[i, j] も 0
s.t. Linking{i in CUSTOMERS, j in FACILITIES}:
    x[i, j] <= y[j];
```

- 数理モデルの記号表記と AMPL の構文が 1 対 1 で直観的に対応 している。
- 添字範囲（‘i in CUSTOMERS‘ 等）で制約を宣言的に記述できるため、バグが混入しにくい。

Python (amplpy) 実装の解説

Python 実装 (1)：データ読込と距離行列生成

```python
import json, math
from amplpy import AMPL

def calculate_distance(loc1, loc2):
    # Timefold quickstart の距離計算式（メートル単位）
    dlat, dlon = loc1[0] - loc2[0], loc1[1] - loc2[1]
    return int(math.ceil(math.sqrt(dlat**2 + dlon**2) * 111000))

# JSONファイルから施設・顧客・重みパラメータを抽出
with open("ロンドン（施設 30・顧客 60）.json", "r", encoding="utf-8") as f:
    data = json.load(f)
problem = data.get("problem", data)
facilities, consumers = problem["facilities"], problem["consumers"]

# 距離行列の事前計算（60 x 30 = 1,800 ペア）
dist_matrix = {
    (c["id"], f["id"]): calculate_distance(c["location"], f["location"])
    for c in consumers for f in facilities
}
```

Python 実装 (2)：AMPL へのデータ投入と求解

```
ampl = AMPL()
ampl.eval("...（先ほどのAMPLモデル文字列）...")

# 1. 集合データの投入
ampl.set["CUSTOMERS"] = [c["id"] for c in consumers]
ampl.set["FACILITIES"] = [f["id"] for f in facilities]

# 2. パラメータデータの投入（辞書形式で直接注入可能）
ampl.param["w_setup"] = 2
ampl.param["w_dist"] = 6
ampl.param["demand"] = {c["id"]: c["demand"] for c in consumers}
ampl.param["capacity"] = {f["id"]: f["capacity"] for f in facilities}
ampl.param["setup_cost"] = {f["id"]: f["setupCost"] for f in facilities}
ampl.param["dist"] = dist_matrix  # タプルキー (i, j) の辞書

# 3. ソルバーに HiGHS を指定して求解実行
ampl.option["solver"] = "highs"
ampl.solve()
```

Python 実装 (3)：結果の抽出と集計

```
# 目的関数値（加重コスト）の取得
obj_val = ampl.get_objective("TotalCost").value()
ampl_soft_score = -int(round(obj_val))

# 開設された施設（y_j > 0.5 となる施設ID）を取得
y_sol = ampl.get_variable("y").get_values().to_dict()
ampl_open_facs = [j for j, val in y_sol.items() if val > 0.5]

# 顧客の割当先施設（x_ij > 0.5 となるペア）を取得
x_sol = ampl.get_variable("x").get_values().to_dict()
ampl_assignment = {i: j for (i, j), val in x_sol.items() if val > 0.5}

# 施設開設コスト合計と実移動距離合計を算出
total_setup = sum(setup_cost[j] for j in ampl_open_facs)
total_dist = sum(dist_matrix[(i, ampl_assignment[i])] for i in c_ids)
```

最適化結果と Timefold 解の比較

最適化結果の比較サマリー

指標	JSON (Timefold)	AMPL (最適解)	改善効果
求解ステータス	メタヒューリスティクス	大域的最適解 (Global)	最適性保証
Soft Score	−1,510,342	−1,467,636	+42,706 改善
総ペナルティ（目的関数）	1,510,342	1,467,636	2.83% 削減
開設施設数	7 施設	7 施設	同数
施設開設コスト合計	313,178	311,028	2,150 削減
顧客移動距離合計 (m)	147,331	140,930	6,401 m 短縮
開設コスト寄与 ($w = 2$)	626,356	622,056	$-4,300$
距離コスト寄与 ($w = 6$)	883,986	845,580	$-38,406$

※ 開設施設数はどちらも 7 施設ですが、AMPL 最適解は開設費用と移動距離の両方を同時に削減しています。

開設施設と顧客割当の差異分析

開設施設の見直し

- 共通開設 (5 施設):
  {10, 11, 15, 22, 25}
- Timefold のみ開設 (閉鎖):
  {13, 26} (開設費: 46,095 / 47,846)
- AMPL 最適解で新設:
  {19, 24} (開設費: 48,631 / 43,160)

顧客割当の変更

- 全 60 顧客中 18 顧客 (30%) の割当先施設が変更。
- より近接した施設への最適再配分が実現。

施設需要の充足量（上限容量 150）

施設 ID	開設費用	JSON 需要	AMPL 需要
10	42,682	90	105
11	46,001	135	135
13	46,095	150	0 (閉鎖)
15	43,421	135	150 (満床)
19	48,631	0	120 (新設)
22	33,084	135	150 (満床)
24	43,160	0	90 (新設)
25	54,049	150	150 (満床)
26	47,846	105	0 (閉鎖)

まとめ

手法の比較とまとめ

- Timefold (局所探索・メタヒューリスティクス):
  - 複雑なビジネスルールや動的変更に柔軟に対応できる。
  - 本問題でも大域的最適解からわずか約 2.8% 差の実用上良好な解を瞬時に探索。
  - ただし、解の「真の最適性」は保証されない。
- AMPL + HiGHS (0-1 整数線形計画法 / 厳密解法):
  - 目的関数・制約条件を明示的に定式化することで、厳密な大域的最適解を算出。
  - 強定式化（Linking Constraint）により、HiGHS でわずか 1 分枝ノード（探索瞬時）で最適解を発見。
  - 開設費用を抑えつつ距離を 6.4 km 短縮し、42,706 ポイントのスコア改善を達成。
- 結論:
  - 実務では、厳密解法（AMPL）でベースラインや大域的最適解を算定し、ヒューリスティクスの評価・改善指標とするアプローチが極めて有効である。